Tuples and Sets in Python

Table of Contents

---

1. Introduction to Tuples and Sets

Python provides several data structures to store collections of data.

So far, we have learned about Lists.

Now we will learn:

- Tuples
- Sets

These data structures help us organize, store, and manage data efficiently.

---

2. Why Tuples and Sets are Important

They help programmers:

- Store multiple values
- Improve performance
- Organize data efficiently
- Prevent unwanted modifications
- Remove duplicate values
- Perform mathematical operations

PART A: TUPLES

---

3. What is a Tuple?

A **Tuple** is an ordered collection of items.

Tuples are similar to lists, but there is one important difference:

👉 Tuples cannot be modified after creation.

This property is called **immutability**.

Syntax of Tuple

tuple_name = (item1, item2, item3)

Example

fruits = ("Apple", "Banana", "Mango")

Output

('Apple', 'Banana', 'Mango')

---

4. Characteristics of Tuples

Ordered

Elements maintain their order.

colors = ("Red", "Green", "Blue")

Immutable

Elements cannot be changed.

colors[0] = "Yellow"

This produces an error.

Allows Duplicates

numbers = (10, 10, 20, 20)

Duplicates are allowed.

Multiple Data Types Allowed

data = ("Python", 100, 99.5, True)

---

5. Creating Tuples

Empty Tuple

empty_tuple = ()

Single Element Tuple

number = (10,)

Important

The comma is mandatory.

Wrong:

number = (10)

This becomes an integer.

Multiple Elements

students = ("Rahul", "Priya", "Amit")

---

6. Accessing Tuple Elements

Use indexing.

Example

fruits = ("Apple", "Banana", "Mango")

print(fruits[0])

Output

Apple

---

7. Tuple Indexing

Index	Value
0	Apple
1	Banana
2	Mango

Negative Indexing

print(fruits[-1])

Output:

Mango

---

8. Tuple Slicing

Slicing extracts multiple elements.

Syntax

tuple[start:end]

Example

numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])

Output

(20, 30, 40)

---

9. Tuple Operations

Concatenation

t1 = (1, 2)
t2 = (3, 4)

print(t1 + t2)

Output:

(1, 2, 3, 4)

Repetition

print((1, 2) * 3)

Output:

(1, 2, 1, 2, 1, 2)

Membership Operator

numbers = (10, 20, 30)

print(20 in numbers)

Output:

True

---

10. Tuple Methods

Tuples have only two methods.

---

count()

Counts occurrences.

numbers = (10, 20, 10, 30)

print(numbers.count(10))

Output:

2

---

index()

Finds position.

numbers = (10, 20, 30)

print(numbers.index(20))

Output:

1

---

11. Advantages and Disadvantages of Tuples

Advantages

- Faster than lists
- Uses less memory
- Secure data storage
- Prevents accidental modifications

Disadvantages

- Cannot add elements
- Cannot remove elements
- Cannot modify elements

PART B: SETS

---

12. What is a Set?

A **Set** is an unordered collection of unique elements.

Sets are mainly used to:

- Remove duplicates
- Perform mathematical operations
- Store unique values

Syntax

set_name = {item1, item2, item3}

Example

fruits = {"Apple", "Banana", "Mango"}

---

13. Characteristics of Sets

Unordered

Elements have no fixed order.

Unique Elements Only

Duplicates are automatically removed.

numbers = {1, 2, 2, 3, 3, 4}

print(numbers)

Output:

{1, 2, 3, 4}

Mutable

Elements can be added or removed.

No Indexing

Sets do not support indexing.

Wrong:

numbers[0]

This causes an error.

---

14. Creating Sets

Empty Set

my_set = set()

Important

my_set = {}

creates a dictionary, not a set.

Set of Numbers

numbers = {10, 20, 30}

Set of Strings

names = {"Rahul", "Amit", "Priya"}

---

15. Adding Elements to Sets

add()

Adds one element.

numbers = {10, 20}

numbers.add(30)

print(numbers)

---

update()

Adds multiple elements.

numbers.update([40, 50, 60])

---

16. Removing Elements from Sets

remove()

numbers.remove(20)

Produces error if value doesn't exist.

---

discard()

numbers.discard(20)

No error if value doesn't exist.

---

pop()

Removes random element.

numbers.pop()

---

clear()

Removes all elements.

numbers.clear()

---

17. Set Operations

Membership Operator

print(20 in {10, 20, 30})

Output:

True

Length

print(len({1, 2, 3}))

Output:

3

---

18. Mathematical Set Operations

Assume:

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

Union

Combines all elements.

print(A | B)

Output:

{1, 2, 3, 4, 5, 6}

Intersection

Common elements.

print(A & B)

Output:

{3, 4}

Difference

Elements present in first set only.

print(A - B)

Output:

{1, 2}

Symmetric Difference

Elements present in either set but not both.

print(A ^ B)

Output:

{1, 2, 5, 6}

---

19. Traversing Sets

Using for loop:

```
fruits = {"Apple", "Banana", "Mango"}

for fruit in fruits:
   print(fruit)
```

---

20. Difference Between Lists, Tuples, and Sets

Feature	List	Tuple	Set
Ordered	Yes	Yes	No
Mutable	Yes	No	Yes
Duplicates Allowed	Yes	Yes	No
Indexing	Yes	Yes	No
Syntax	[]	()	{}

---

21. Common Programs Using Tuples and Sets

Program 1: Count Elements in Tuple

```
numbers = (10, 20, 30, 40)

print(len(numbers))
```

Output:

4

Program 2: Find Maximum Value

```
numbers = (10, 20, 50, 40)

print(max(numbers))
```

Output:

50

Program 3: Remove Duplicates Using Set

```
numbers = [10, 20, 10, 30, 20]

unique_numbers = set(numbers)

print(unique_numbers)
```

Output:

{10, 20, 30}

Program 4: Common Elements Between Two Sets

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A & B)
```

Output:

{3, 4}

---

22. Real-Life Applications

Tuples

Used in:

- GPS Coordinates
- Database Records
- Employee Information
- Fixed Configuration Data
- Banking Systems

Example:

```
location = (18.5204, 73.8567)
```

Sets

Used in:

- Removing duplicate records
- Attendance systems
- Unique user IDs

- Search engines
- Recommendation systems
- Data analysis

Example:

students = {"Rahul", "Priya", "Rahul", "Amit"}

Output:

{'Rahul', 'Priya', 'Amit'}

---

23. Conclusion

Tuples and Sets are powerful Python data structures designed for different purposes.

Tuples

- Ordered
- Immutable
- Faster
- Secure

Sets

- Unordered
- Unique elements only
- Excellent for mathematical operations
- Useful for removing duplicates

Understanding Tuples and Sets is essential because they are widely used in Data Structures, Data Science, Artificial Intelligence, Database Systems, and Software Development. They provide efficient ways to store and manage data while solving real-world programming problems.